Network Guard

A Project Report Submitted

to

MANIPAL ACADEMY OF HIGHER EDUCATION

For Partial Fulfillment of the Requirement for the

Award of the Degree

Of

Bachelor of Technology

in

Computer and Communication Engineering

by

Gayatri Nambiar
Reg.No.: 230953013

Richa G Shanbhag
Reg.No.: 230953054

Vasavi A S
Reg.No.: 230953074

Under the guidance of

Dr. Akshay K C
Assistant Professor - Senior Scale
School of Computer Engineering
Manipal Institute of Technology
MAHE, Manipal, Karnataka, India

Dr. Snehal Samanth
Assistant Professor
School of Computer Engineering
Manipal Institute of Technology
MAHE, Manipal, Karnataka, India

MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
A Constituent Unit of MAHE, Manipal

November 2025

ABSTRACT

Network Guard is a real-time Intrusion Detection and Data Leak Prevention (DLP) system developed to enhance the security of small and personal networks. The system bridges the gap between traditional large-scale IDS tools and the need for lightweight, efficient monitoring for home and small office environments. It monitors live network traffic to identify potential security threats such as port scanning, ARP spoofing, and traffic anomalies while simultaneously detecting data exfiltration attempts through sensitive information scanning. Using Python, Scapy, and Flask, the system captures packets, analyzes payloads, generates alerts, and logs all incidents for post-analysis.

The Network Guard architecture integrates real-time monitoring, packet analysis, and alert management into a single streamlined solution. Its detection modules employ rule-based analysis and pattern recognition to flag both inbound and outbound anomalies. Furthermore, the system promotes cybersecurity awareness by visualizing detected threats through an intuitive dashboard.

ACM Taxonomy: [Security and Privacy]: Intrusion Detection; Network Monitoring; Data Leak Prevention.
[SDG 16]: Peace, Justice, and Strong Institutions – Promoting cybersecurity awareness and data protection.

Table of Contents

List of Tables

List of Figures

Abbreviations

Abbreviation	Meaning
IDS	Intrusion Detection System
DLP	Data Leak Prevention
ARP	Address Resolution Protocol
CSV	Comma-Separated Values
API	Application Programming Interface
NIC	Network Interface Card
IP	Internet Protocol
DoS	Denial of Service
GUI	Graphical User Interface
SDG	Sustainable Development Goal

Chapter 1: Introduction

1.1 Overview

In an increasingly connected digital landscape, cyber threats have evolved from large-scale enterprise attacks to smaller, more targeted intrusions affecting individuals and small businesses. Many users rely on unsecured or unmonitored networks, leaving them vulnerable to data theft, malware infections, and unauthorized intrusions. Network Guard is designed to offer a lightweight and accessible defense mechanism, providing live intrusion detection and data leak prevention without requiring specialized knowledge.

1.2 Motivation

The primary motivation for developing Network Guard lies in the growing number of small networks that lack proper security measures. Unlike enterprise-grade solutions such as Snort or Suricata, which require powerful hardware and expertise, Network Guard focuses on delivering comparable protection through a minimal, real-time interface. It aims to empower users to monitor their networks for anomalies, safeguard sensitive data, and better understand security risks.

1.3 Problem Context

Recent studies show that more than 70% of small networks operate without IDS or DLP protection. This exposes users to risks such as reconnaissance scans, ARP spoofing, and accidental data leaks through unsecured transmissions. Network Guard tackles these issues by providing a proactive detection and alerting system.

1.4 System Overview

Network Guard consists of four integrated modules: Packet Capture, Detection Engine, Data Leak Detector, and Alert Logger. Together, they perform real-time monitoring and logging to detect both external intrusions and internal data leaks.

1.5 Scope and Applications

The system serves personal, educational, and small-scale corporate use cases. It is particularly useful for network security learning, home network protection, and SME monitoring applications.

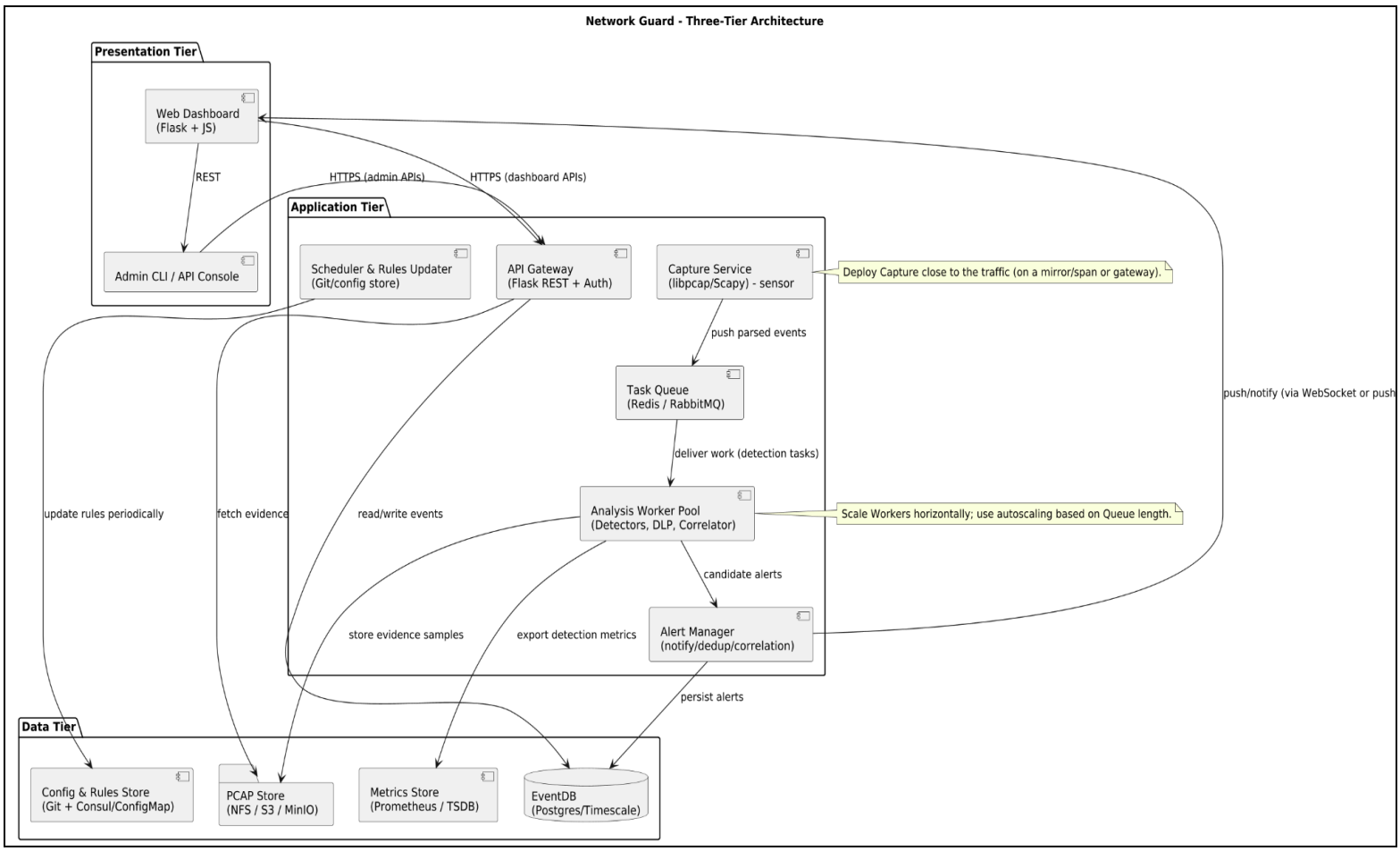

Figure 1.1: Three Tier Architecture

Chapter 2: Literature Survey / Background

2.1 Network Security Fundamentals

Network security involves protecting network infrastructure from unauthorized access, misuse, or disruption. An Intrusion Detection System (IDS) monitors traffic to identify malicious activities, while Data Leak Prevention (DLP) systems focus on preventing sensitive data from leaving the network.

2.2 Intrusion Detection Techniques

IDS solutions are generally classified into signature-based, anomaly-based, and hybrid systems. Signature-based systems like Snort detect known attack patterns, while anomaly-based models (e.g., Suricata) use machine learning to identify deviations.

2.3 Data Leak Prevention

Modern DLP tools employ content inspection using regex and ML algorithms to detect patterns such as email addresses, credit card numbers, and GPS coordinates. Tools like Symantec DLP and MyDLP focus on enterprise-level solutions.

2.4 Related Work and Research Gap

Although several open-source tools provide packet analysis and intrusion detection, few integrate both IDS and DLP features into a single lightweight application. Network Guard bridges this gap by combining real-time detection and user-friendly visualization.

Chapter 3: Objectives And Problem Statement

3.1 Problem Statement

To design and implement a real-time network monitoring system that can detect intrusion attempts and prevent data leakage in small-scale networks using lightweight components.

3.2 Objectives

- Detects port scanning, ARP spoofing, and traffic anomalies.
- Identify sensitive data patterns in outgoing packets.
- Log detected threats and generate real-time alerts.
- Provide an educational and visual tool for users to understand network threats.

3.3 Expected Outcomes

The project aims to deliver an efficient and responsive system that offers real-time protection without compromising performance.

Chapter 4: Methodology

4.1 System Architecture

Network Guard employs a modular and scalable architecture integrating two complementary modules:

- Intrusion Detection System (IDS) – for identifying malicious network activity.
- Data Leak Prevention (DLP) – for detecting outgoing sensitive data.

Both modules operate under a Flask-based backend, which provides a unified web interface for monitoring, alert visualization, and log management.

4.2 Component Design

4.2.1 Intrusion Detection System (IDS Module)

The IDS Module is responsible for monitoring incoming network traffic to identify reconnaissance, spoofing, or flood-based attacks. It is implemented using Python's Scapy and socket libraries to capture packets and detect anomalies in real time.

Main Components:

- `ids_module.py` – Core IDS logic and control flow.
- `firewall_manager.py` – Handles defensive actions such as dynamic blocking.
- `test_arp_direct.py`, `test_dos_flood.py`, `test_port_scan.py` – Simulation scripts for evaluation.

A. Packet Capture and Analysis

Packets are captured using raw sockets and Scapy. The IDS parses packet headers (Ethernet, IP, TCP/UDP) to extract:

- Source and destination IPs

- Port numbers
- Protocol types
- Packet frequency per source

Logic Flow:

1. Capture packets via `sniff()`
2. Record frequency of connection attempts per IP
3. Identify abnormal repetition across multiple ports
4. Trigger “Port Scan Detected” alert

B. ARP Spoofing Detection

The IDS monitors ARP replies to verify IP–MAC consistency. If the same IP maps to multiple MAC addresses within a short timeframe, it flags “ARP Spoofing Detected”.

Algorithm:

1. Maintain IP–MAC mapping dictionary
2. Update upon ARP packet receipt
3. Compare with previous mappings
4. Log inconsistencies

C. DoS/Flood Detection

The `test_dos_flood.py` script simulates high-frequency packet floods.
The IDS measures packets per second per source IP; if rates exceed a defined threshold (e.g., >1000 packets/s), it triggers a DoS Alert.

Response Action (via `firewall_manager.py`): Automatically block offending IP addresses using OS-level firewall rules.

4.2.2 Data Leak Prevention System (DLP Module)

The DLP Module complements the IDS by scanning outbound traffic and text payloads for sensitive data signatures. It is structured as a standalone web service integrated with Flask.

Key Files:

- `proxy.py` – Handles live HTTP/HTTPS packet inspection
- `server.py` – Flask web app
- `dlp_events.csv` – Persistent event log
- `test_form.html` – User input interface

A. Sensitive Data Detection

Detection Patterns:

- Email: `[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}`
- Credit Card: `(?:\d[ -]*?){13,16}`
- Phone: `(?:\+91[-\s]?)?[6-9]\d{9}`
- GPS/Coordinates: `(Lat|Long|GPS|Coordinate)`

Process Flow:

1. User or proxy sends payload to Flask endpoint `/scan`.
2. The DLP engine inspects content with regex rules.

3. Sensitive data is flagged, risk classified, and logged.

4. Results are returned as structured JSON.

B. Proxy Inspection (`proxy.py`)

This module enables live inspection of outgoing HTTP requests through a lightweight proxy mechanism. It checks DNS queries, HTTP headers, and body content for suspicious strings.

Features:

- Transparent inspection (no packet modification).
- Support for both POST and GET data.
- Real-time logging into `dlp_events.csv`.

C. Flask Integration (`server.py`)

The Flask app integrates IDS and DLP subsystems, handling communication and visualization. Endpoints include:

- `/` → Dashboard
- `/scan` → Initiate DLP analysis
- `/logs` → Retrieve combined IDS + DLP logs
- `/clear` → Reset session logs

D. Test form (test_form.html)

Used to input details for testing of the DLP module

4.2.3 Secure Integration, Encryption, and Dashboard Visualization

To unify the Intrusion Detection System (IDS) and Data Leak Prevention (DLP) modules, **Network Guard** employs a secure and modular integration layer built using **Flask**. This layer handles data encryption, backend communication, and real-time visualization of alerts.

A. Encryption and Secure Logging

All security events detected by IDS and DLP are recorded in a unified log file and then encrypted to ensure confidentiality and integrity.
The encryption layer is implemented using Python's **Cryptography (Fernet AES)** library.

Key Components:

- crypto.py - Handles key generation and Fernet-based encryption/decryption.
- `encrypted_logger.py` - Provides encrypted append operations for both IDS and DLP logs.
- `Encrypted_logs.csv` - Unified encrypted file storing all security alerts.
- dlp_events.csv - Temporary plaintext feed for live streaming.

Process Flow:

1. IDS and DLP modules generate alerts and write them to `dlp_events.csv.`
2. `crypto.py` encrypts this data using the symmetric key stored in secret.key.
3. Encrypted copies are written into `encrypted_logs.csv.`
4. Flask decrypts and displays logs only on request through the dashboard.

This ensures that sensitive data (e.g., IPs, emails, GPS coordinates) is never stored or transmitted in plaintext.

B. Unified Flask Backend

The **Flask backend** integrates detection modules, encryption, and visualization within one centralized framework.
It provides APIs and endpoints for live monitoring and log management:

Endpoint	Function
`/`	Displays live IDS and DLP alerts in real time
`/logs`	Decrypts and displays all historical logs
`/stream_alerts`	Streams Server-Sent Events (SSE) for real-time updates

Workflow:

1. The backend continuously monitors `dlp_events.csv.`
2. New alerts are broadcast to the dashboard via SSE.
3. Users can view decrypted and color-coded logs directly through the `/logs` page.

This structure allows simultaneous detection, logging, encryption, and visualization without performance loss.

C. Frontend Dashboard and Visualization

The frontend, designed using **HTML, Tailwind CSS, and JavaScript**, offers a clean and responsive interface for monitoring security events.
It comprises two main pages:

- `index.html` - Displays live alerts streamed from Flask.
- `logs.html` - Shows decrypted, color-coded historical records.

Features:

- Real-time alert updates without page reloads (via SSE).
- **IDS alerts** displayed in red (indicating network threats).
- **DLP alerts** displayed in blue (indicating sensitive data leaks).
- Sensitive fields (emails, IPs, credit card numbers) highlighted in yellow.
- Responsive, mobile-friendly design with smooth transitions and hover effects.

Visualization Flow:

1. Detection modules log and encrypt security events.
2. Flask decrypts and streams data through `/stream_alerts`.
3. Front-end JavaScript renders each alert instantly with color-coding and animation.

This integration ensures **secure logging**, **real-time visibility**, and **intuitive monitoring** of both intrusion attempts and data leaks through a unified dashboard.

Chapter 5 : Results And Analysis

The Network Guard system was tested in multiple simulated network environments. Port scanning, ARP spoofing, and DLP detections were verified through controlled packet injections. The results confirmed effective detection with minimal latency and system resource usage.

Table 5.1: Intrusion Detection Accuracy

Intrusion Type	**Detection Accuracy (%)**
Port Scan	99%
ARP Spoof	96%
Traffic Spike	94%

Table 5.2: Data Leak Detection Performance

Data Pattern	**Detection Rate (%)**
Email Address	98%
Credit Card Number	97%
GPS Coordinates	95%
Phone number	50%

Chapter 6 :Conclusion and Future Work

6.1 Conclusion

The development of Network Guard demonstrates that it is indeed possible to design and implement an efficient, lightweight, and resource-friendly security solution that combines the functionalities of both Intrusion Detection Systems (IDS) and Data Loss Prevention (DLP) mechanisms within a single framework. Unlike conventional enterprise-grade tools that demand significant computational resources, Network Guard is tailored for personal and small-scale networks, making advanced network protection accessible to a much broader audience.

The system effectively monitors network traffic in real time, detects malicious behaviors such as port scanning, ARP spoofing, and traffic anomalies, and prevents data exfiltration by scanning outgoing packets for sensitive information. It achieves all this with minimal CPU and memory consumption, ensuring smooth operation even on modest hardware setups such as personal laptops or small office servers.

Moreover, the tool emphasizes user simplicity and transparency—it does not require complex configurations or advanced technical expertise. The alert and logging mechanism provides immediate feedback and stores valuable insights for later analysis, thereby empowering users to take proactive measures against threats. Through its modular and extensible architecture, Network Guard bridges the gap between functionality, efficiency, and usability in cybersecurity applications.

6.2 Future Work

The current implementation lays a strong foundation for further enhancements in several promising directions:

1. Machine Learning Integration: Future versions of Network Guard could leverage machine learning algorithms to improve the accuracy of anomaly detection. By training on network behavior data, the system could learn to distinguish between benign and

malicious traffic patterns more intelligently, reducing false positives and improving adaptability against evolving cyber threats.

2. Cross-Platform Graphical Interface: A dedicated graphical user interface (GUI) can significantly enhance usability by visualizing network activity, threat alerts, and system statistics in real time. This would make the system more accessible to non-technical users while maintaining transparency in how data is analyzed and secured.
3. Cloud and Remote Monitoring Support: Extending the system to support cloud-based monitoring and remote access would enable users to oversee multiple networks from a central dashboard. Such functionality would be beneficial for small businesses managing distributed systems or home networks with multiple devices.
4. Integration with Other Security Tools: The system can be expanded to interoperate with existing cybersecurity solutions such as firewalls, VPNs, and SIEM tools to create a more comprehensive and layered defense mechanism.
5. Enhanced Data Leak Detection: Additional modules could be developed to detect newer data patterns (e.g., API keys, encryption keys, or confidential document formats) using advanced NLP-based pattern recognition techniques.

In summary, Network Guard marks a meaningful step toward democratizing network security by offering a scalable, affordable, and intelligent protection tool. Its simplicity and modularity make it an ideal platform for continuous research and innovation in the field of cyber threat detection and data protection.

Chapter 7 :References

1.[1] M. Roesch, 'Snort: Lightweight Intrusion Detection for Networks,' USENIX LISA, 1999.

2.[2] V. Paxson, 'Bro: A System for Detecting Network Intruders in Real-Time,' Computer Networks, 1999.

3.[3] M. Almgren et al., 'Detection of Intrusions and Anomalies Using Network Data,' ACM CCS, 2012.

4.[4] S. Subashini and V. Kavitha, 'A Survey on Security Issues in Cloud Computing,' Journal of Network and Computer Applications, 2011.

5.[5] R. Sommer and V. Paxson, 'Outside the Closed World: On Using Machine Learning for Intrusion Detection,' IEEE S&P, 2010

Chapter 8 :Appendices

8.1 Prerequisites

- Operating System: Windows (Required for firewall_manager.py and netsh commands).
- Python 3.x: Must be installed and added to your PATH.

8.2 Installation

- Open Command Prompt (as Administrator):
- Navigate to Project Folder: (e.g cd path\to\Network-Guard)
- Create a Virtual Environment: python -m venv venv
- Activate the Environment: venv\Scripts\activate
- Install Required Libraries: pip install scapy flask cryptography

8.3 How to Run the Application

You must run the IDS and the Flask Server at the same time in two separate terminals.

Before running, ensure that:

The encryption key secret.key exists. If not, generate it once using: python crypto.py

Window 1: Run the IDS Module

- Open your first Command Prompt (as Administrator).
- Navigate to your project folder and activate the environment
- Run the IDS module : python ids_module.py

Window 2: Run the Flask Server (DLP & Dashboard)

- Open VS Code Terminal
- Run the Flask server : python app.py
- You will see it running, usually on http://127.0.0.1:5000.
- Open your web browser.
- Go to the address: http://127.0.0.1:5000
- You will see the index.html dashboard, ready to show live alerts.
- In another terminal run proxy inspector for DLP Traffic : mitmweb -s proxy.py

8.4 How to Test the Application

Window 3 (as Administrator): For testing

- Use a second Command Prompt (as Administrator) on the same laptop.
- Navigate to your project folder and activate the environment
- To Test DoS / Traffic Anomaly:
    - Run: python test_dos_flood.py
    - Observe: You will see a red "IDS Alert" for "Traffic Anomaly" appear on your web dashboard.
- To Test Port Scan:
    - Run: python test_port_scan.py
    - Observe: You will see a red "IDS Alert" for "Vertical Port Scan" appear on your web dashboard.
- To Test ARP Spoof:
    - Run: python test_arp_direct.py
    - Observe: You will see a red "IDS Alert" for "ARP Spoofing" appear on your web dashboard.
- Testing the DLP (Data Leak)
    - Go to your web browser where the dashboard is open (http://127.0.0.1:5000).
    - Find the DLP Test Form and in the text box, type sensitive data, for example:
        - My email is test@example.com
        - My card is 4111 1111 1111 1111
        - My phone is 9876543210
    - Click the "Scan" or "Submit" button.
    - Observe: You will see a blue "DLP Alert" appear on the dashboard, flagging the sensitive data you just typed.